Diabetes Classification using Single Layer Perceptron

Sameet Asadullah
Adelaide University
Adelaide, South Australia, Australia
sameet.asadullah@student.adelaide.edu.au

Abstract

Diabetes ranks among the top causes of sickness worldwide. It is essential to detect diabetes as soon as possible. This research introduces a perceptron-based approach to classify diabetes data using neural networks to improve precision and effectiveness. We utilized the model for training and evaluation at various learning rates and different optimizers. We attained an accuracy of 73.38%, with the optimal learning rate and loss function being 0.001 and Adam respectively.

1. Introduction

Neural Networks have played a crucial role in tackling various challenges, including medical diagnosis and predicting energy demand. Timely detection of illnesses is crucial for overall well-being. This is why our focus is on utilizing a basic form of Neural Networks, specifically a Single Layer Perceptron (SLP) [1], to classify diabetes. We conducted experiments with different learning rates and optimizers to achieve the highest accuracy possible using SLP. The original dataset and its details can be accessed on Kaggle [2]. The dataset contains several attributes, which are used to predict diabetes. However, for training the SLP, we utilized preprocessed scaled data available on the website of the National Taiwan University [3].

Significant research has already been conducted on addressing this issue [4, 5, 6], resulting in an impressive accuracy of over 95%. However, the previous studies utilized a more complex neural network known as Multi-Layer Perceptron (MLP) [7]. By building upon these investigations, we have implemented the same preprocessing techniques and evaluation criteria in our study. Our research introduces a technique that employs a single-layer perceptron to categorize diabetes data, tapping into the capabilities of artificial neural networks.

2. Method

This segment will focus on the techniques used in the research project. Afterward, we will present an analysis of the dataset and approach in the following subsections.

2.1. Dataset

The research utilizes a dataset from the Pima Indian community. The dataset contains 9 attributes, including 8 features and 1 class label. The histogram for analyzing data distribution is given below.

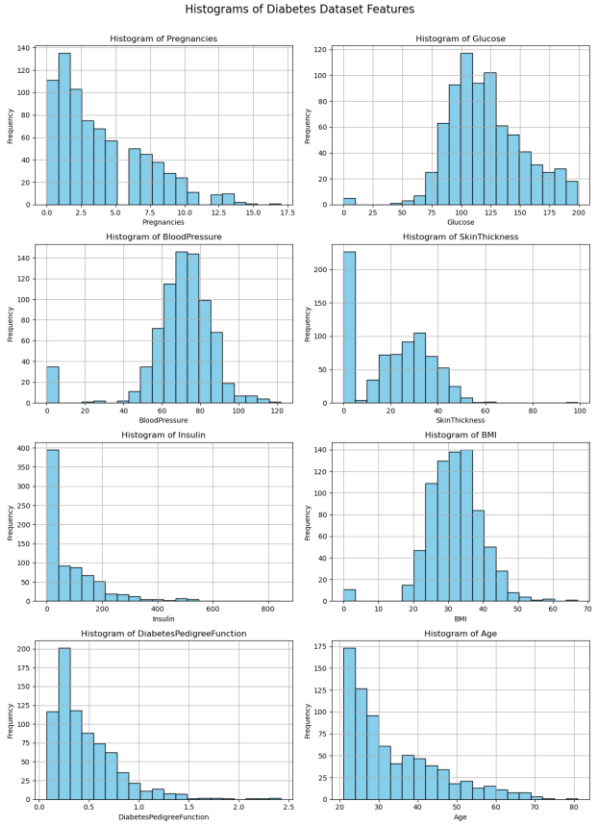


Figure 1: Histogram of each feature

To understand the label distribution of dataset, we draw a pie chart which you can see below and notice that there are 65.1% and 34.9% non-diabetic and diabetic persons respectively. Training the model as it is on this distribution level will not produce good results because it will be biased towards non-diabetic persons. So, we will

resolve this issue by implementing SMOTE technique [8] to balance the data before training.

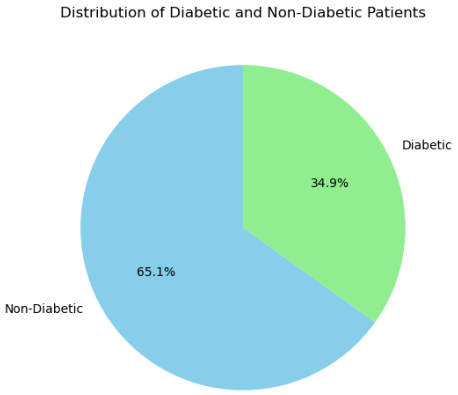


Figure 2: Pie chart of Diabetic vs Non-Diabetic Patients

Upon further analyzation of the dataset, we also realize that it requires standardization [9] because the standard deviation and mean of the dataset is not 1 and 0 respectively. We use StandardScaler [10] to standardize the data. The scaled dataset was not standardized because it was normalized [11]. We can only have one and not both altogether. In case of Linear models i.e., Single Layer Perceptron, standardization is more preferred because the model is sensitive to input feature scales.

2.2. Algorithm

The Single-Layer Perceptron (SLP) [1] is among the most basic types of artificial neural networks (ANNs). It functions as a linear classifier by computing a weighted sum of its input features and then comparing this sum to a threshold to generate an output. The output is based on the activation function that we choose. To introduce non-linearity in neural networks, we use activation functions. Additionally, they are also used in determining whether a neuron should be activated or not during training [12]. The basic components of an SLP are depicted in Figure below:

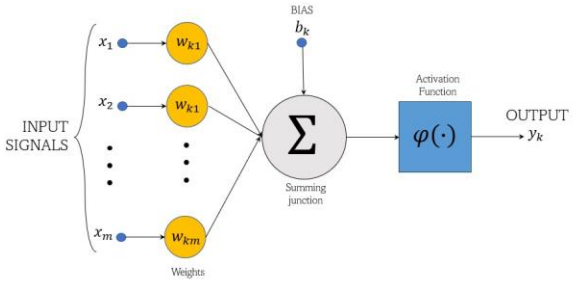


Figure 3: Single Layer Perceptron Components

The input signals $x1, x2, \dots, x_m$ undergo multiplication with corresponding weights $w_{k1}, w_{k2}, \dots, w_{km}$. After this, the weighted inputs are added together, along with a bias b_k , at the summing junction. The outcome is then subjected to an activation function $\varphi(.)$ to generate the neuron's output, which introduces non-linearity to the model. Despite its simplicity, the SLP is widely applicable in various fields. The core equation behind the SLP is:

$$y_k = \varphi(\sum_{i=1}^{m} x_i w_{ki} + b_k)$$

As this is a binary classification problem, we will be using sigmoid activation function on the output layer because its result ranges from 0 to 1, making it ideal for predicting binary classes [12]. The equation of sigmoid activation function is:

$$\varphi(z) = \frac{1}{1 + e^{-z}}$$

Calculating the loss is our final step in forward propagation. In a model, the loss function analyses how many model's predictions match the real values and tells us a value of overall error in the model. It calculates the error between predicted results and real targets. Then we use the output of loss function to further guide the model so that it could learn better and converge towards the actual targets. As this is a binary classification problem, we will be using Binary Cross Entropy (BCE) Loss [13] which is specifically designed for these kinds of problems. Its equation is:

$$L = -\frac{1}{N}\sum_{i=1}^{N}(y_{true}^{(i)} \log(y_{pred}^{(i)}) + (1 - y_{true}^{(i)}) \log(1 - y_{pred}^{(i)}))$$

There are two steps involved while training the model, forward propagation and back propagation. Forward propagation is what we explained above and incase of backward propagation there are a few more steps included. The first one is to calculate the error term with the following equation:

$$\delta^{(i)} = y_{pred}^{(i)} - y_{true}^{(i)}$$

After calculating the error term, we update the weights and biases using and optimizer. We will be testing with two optimizers Stochastic Gradient Descent (SGD) [14] and Adam [15] because these are the ones famous for better and faster convergence.

2.3. Implementation

After going through and understanding the algorithm in detail, it is pretty simple to implement it. We take the scaled dataset from Kaggle, balance it using SMOTE [8], and then standardize [9] it. Afterwards, using the above methods we train the Single Perceptron Layered model on our preprocessed dataset. The training process involves multiple iterations with varying parameters to conduct thorough experimentation. Following this, we employ diverse evaluation techniques to determine the optimal performance for our specific use case and dataset.

3. Experimental Analysis

We conduct experiments on various parameters to determine the optimal performance on our dataset. In our testing, we are trying out various learning rates, including 0.001, 0.01, and 0.1, and also examining different optimizers such as Stochastic Gradient Descent (SGD) [14] and Adam [15].

We analyze the model's performance through visual representations such as the loss curve and accuracy plot which is the most effective approach. After plotting these visualizations, we have determined that our model achieves the highest performance with a Learning Rate of 0.001 and Adam Optimizer, resulting in an accuracy of 73.38%.

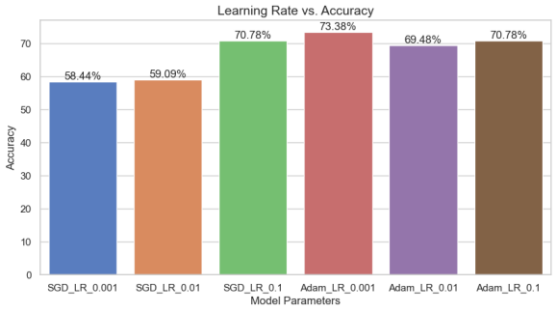


Figure 4: Accuracy bars of models on different parameters

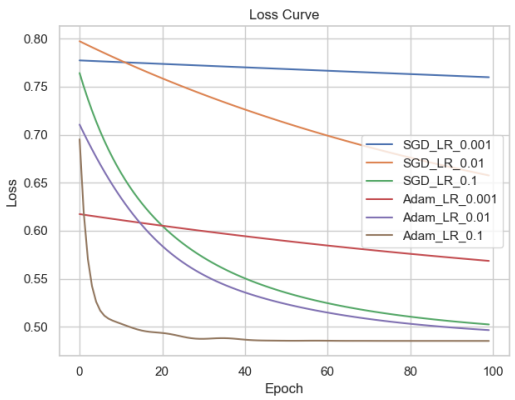


Figure 5: Loss curves of models on different parameters

To further analyze our binary classification model, we use Confusion Matrix and Receiver Operating Characteristic (ROC) curve [16]. In confusion matrix, the predictions made by a model are analyzed to show how they compare to the actual outcomes, essentially indicating the level of confusion in the model.

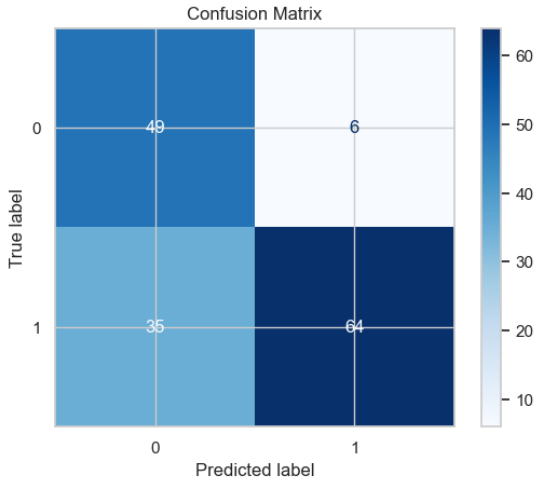


Figure 6: Confusion Matrix of the final model

Examining the confusion matrix reveals that the model has difficulty when the actual label is 1 but it predicts it as 0. This indicates that it has difficulty recognizing cases where a person has diabetes and instead categorizes them as non-diabetic. This might be due to the original dataset containing more non-diabetic individuals. Despite our efforts to balance the data, it is difficult to achieve complete accuracy.

Moving on to the ROC curve, it shows us the model's ability to differentiate between classes at various decision thresholds, offering understanding into the model's performance at different thresholds and the trade-off between true positive rate (TPR) and false positive rate (FPR).

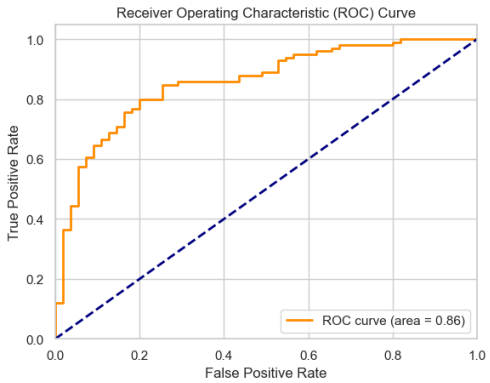


Figure 7: ROC curve of the final model

To check how well the model is doing overall, we calculate the area under the Curve. A higher area means the model performs better. For the Single Layer Perceptron model, we got an AUC of 0.86, which is a good result.

4. Conclusion

In our project, we have utilized a Single Layer Perceptron to determine whether an individual has diabetes. We've discovered that achieving high accuracy doesn't always require a complex model. By appropriately preprocessing the data and selecting optimal hyperparameters for the model, we can make significant improvements in both accuracy and training speed. Using just one layer in the model with a learning rate of 0.001, along with Adam as the optimizer and Binary Cross Entropy as a loss function, we have attained a 73.38% accuracy.

In future work, the confusion matrix shows that the model struggles with correctly classifying individuals with diabetes, likely due to data imbalance. While SMOTE has already been used, we can explore other techniques, such as ADASYN, Random Over-Sampling, or BorderlineSMOTE [17], to further address the imbalance and improve model performance. Once the data is balanced and accuracy improves, we can also experiment with adding additional layers to the model, carefully ensuring we enhance accuracy without overfitting.

5. Code

The code and dataset used for this research is available here.

6. References

[1] A. David, "Single Layer Perceptron and Activation Function," Medium, 2021.

[2] UCI Machine Learning, "Pima Indians Diabetes Database," Kaggle, 2016.

[3] National Taiwan University, "Pima Indian Diabetes," National Taiwan University, Taiwan.

[4] J. S. N. Y. M. R. C. R. R. V. Sivasankari S S, "Classification of Diabetes using Multilayer Perceptron," in *IEEE*, Ballari, India, 2022.

[5] H. M. E.-B. Abeer El-Sayyid El-Bashbishy, "Pediatric diabetes prediction using deep learning," *Scientific Reports*, p. 20, 2024.

[6] S. A. Huma Naz, "Deep learning approach for diabetes prediction using PIMA Indian dataset," *J Diabetes Metab Disord*, vol. 19, no. 1, p. 391–403, 2020.

[7] C. Bento, "Multilayer Perceptron Explained with a Real-Life Example and Python Code: Sentiment Analysis," Towards Data Science, 2021.

[8] K. W. B. L. O. H. W. P. K. N. V. Chawla, "SMOTE: Synthetic Minority Over-sampling Technique," *Journal Of Artificial Intelligence Research*, vol. 16, pp. 321-357, 2002.

[9] G. D. C. R. M. C. Lucas B.V. de Amorim, "The choice of scaling technique matters for classification performance," *Applied Soft Computing*, vol. 133, 2023.

[10] A. Upadhyay, "StandardScaler and Normalization with code and graph," Medium, 2020.

[11] S. Jaiswal, "What is Normalization in Machine Learning? A Comprehensive Guide to Data Rescaling," Datacamp, 2024.

[12] S. SHARMA, "Activation Functions in Neural Networks," Medium, 2017.

[13] D. Godoy, "Understanding binary cross-entropy / log loss: a visual explanation," Towards Data Science, 2018.

[14] A. V. Srinivasan, "Stochastic Gradient Descent — Clearly Explained !!," Towards Data Science, 2019.

[15] D. Wei, "Demystifying the Adam Optimizer in Machine Learning," Medium, 2024.

[16] Joleen, "Evaluating Classification Models: Understanding the Confusion Matrix and ROC Curves," Statistically Relevant, 2023.

[17] H. Aguiar, "What Is Imbalanced Data and How to Handle It?," TURINTECH.

[18] J. E. W. D. W. K. R. J. Jack W. Smith, "Using the ADAP Learning Algorithm to Forecast the Onset of Diabetes Mellitus," *Proc Annu Symp Comput Appl Med Care*, p. 261–265, 1988.

[19] Evidently AI Team, "Accuracy vs. precision vs. recall in machine learning: what's the difference?," Evidently AI.